

Hands-on Lab: Build a Streaming ETL Pipeline using Kafka

You are a data engineer at a data analytics consulting company. You have been assigned to a project that aims to de-congest the national highways by analyzing the road traffic data from different toll plazas. As a vehicle passes a toll plaza, the vehicle's data like *vehicle_id*, *vehicle_type*, *toll_plaza_id*, and *timestamp* are streamed to Kafka. Your job is to create a data pipe line that collects the streaming data and loads it into a database.



Project scenario

You are a data engineer at a data analytics consulting company. You have been assigned to a project that aims to de-congest the national highways by analyzing the road traffic data from different toll plazas. As a vehicle passes a toll plaza, the vehicle's data like `vehicle_id`, `vehicle_type`, `toll_plaza_id`, and `timestamp` are streamed to Kafka. Your job is to create a data pipe line that collects the streaming data and loads it into a database.

Objectives

In this assignment, you will create a streaming data pipe by performing these steps:

- Start a MySQL database server
- Create a table to hold the toll data

- Start the Kafka server
- Install the Kafka Python driver
- Install the MySQL Python driver
- Create a topic named toll in Kafka
- Download streaming data generator program
- Customize the generator program to stream to toll topic
- Download and customize streaming data consumer
- Customize the consumer program to write into a MySQL database table
- Verify that streamed data is being collected in the database table

Exercise 1: Download and extract Kafka

1. Download Kafka by running the command below.

```
1 wget https://archive.apache.org/dist/kafka/3.7.0/kafka_2.12-3.7.0.tgz
```

2. Extract Kafka from the zip file by running the command below.

```
1 tar -xzf kafka_2.12-3.7.0.tgz
```

Note: This command creates a directory named `kafka_2.12-3.7.0` in the current directory.

Exercise 2: Configure KRaft and start server

1. Change to the `kafka_2.12-3.7.0` directory.

```
1 cd kafka_2.12-3.7.0
```

2. Generate a cluster UUID that will uniquely identify the Kafka cluster.

```
1 KAFKA_CLUSTER_ID="$(bin/kafka-storage.sh random-uuid)"
```

Note: The new cluster id generated will be used by the KRaft controller.

3. KRaft requires the log directories to be configured. Run the following command to configure the log directories passing the cluster id.

```
1 bin/kafka-storage.sh format -t $KAFKA_CLUSTER_ID -c config/kraft/server.properties
```

```
theia@theiadocker-sannyjrgarin:/home/project$ wget https://downloads.apache.org/kafka/3.8.0/kafka_2.13-3.8.0.tgz
--2024-11-19 04:32:37-- https://downloads.apache.org/kafka/3.8.0/kafka_2.13-3.8.0.tgz
Resolving downloads.apache.org (downloads.apache.org)... 135.181.214.104, 88.99.208.237, 135.181.214.104, ...
Connecting to downloads.apache.org (downloads.apache.org)|135.181.214.104|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 120735482 (115M) [application/x-gzip]
Saving to: 'kafka_2.13-3.8.0.tgz'

kafka_2.13-3.8.0.tgz      100%[=====>] 115.14M  3.44MB/s   in 28s

2024-11-19 04:33:05 (4.16 MB/s) - 'kafka_2.13-3.8.0.tgz' saved [120735482/120735482]

theia@theiadocker-sannyjrgarin:/home/project$ tar -xzf kafka_2.12-3.8.0.tgz
tar (child): kafka_2.12-3.8.0.tgz: Cannot open: No such file or directory
tar (child): Error is not recoverable: exiting now
tar: Child returned status 2
tar: Error is not recoverable: exiting now
theia@theiadocker-sannyjrgarin:/home/project$ ls
ETL_toll_data.py  kafka_2.13-3.8.0.tgz  ooxwv-docker_hadoop
theia@theiadocker-sannyjrgarin:/home/project$ tar -xzf kafka_2.13-3.8.0.tgz
theia@theiadocker-sannyjrgarin:/home/project$ cd kafka_2.13-3.8.0
theia@theiadocker-sannyjrgarin:/home/project/kafka_2.13-3.8.0$ KAFKA_CLUSTER_ID="$(bin/kafka-storage.sh random-uuid)"
theia@theiadocker-sannyjrgarin:/home/project/kafka_2.13-3.8.0$ bin/kafka-storage.sh format -t $KAFKA_CLUSTER_ID -c config/kraft/server.properties
Formatting /tmp/kraft-combined-logs with metadata.version 3.8-IV0.
theia@theiadocker-sannyjrgarin:/home/project/kafka_2.13-3.8.0$
```

4. Now that KRaft is configured, you can start the Kafka server by running the following command.

```
1 bin/kafka-server-start.sh config/kraft/server.properties
```

Note: You can be sure that the Kafka server started there is information generated that the server started successfully along with some additional messages, such as log loaded.

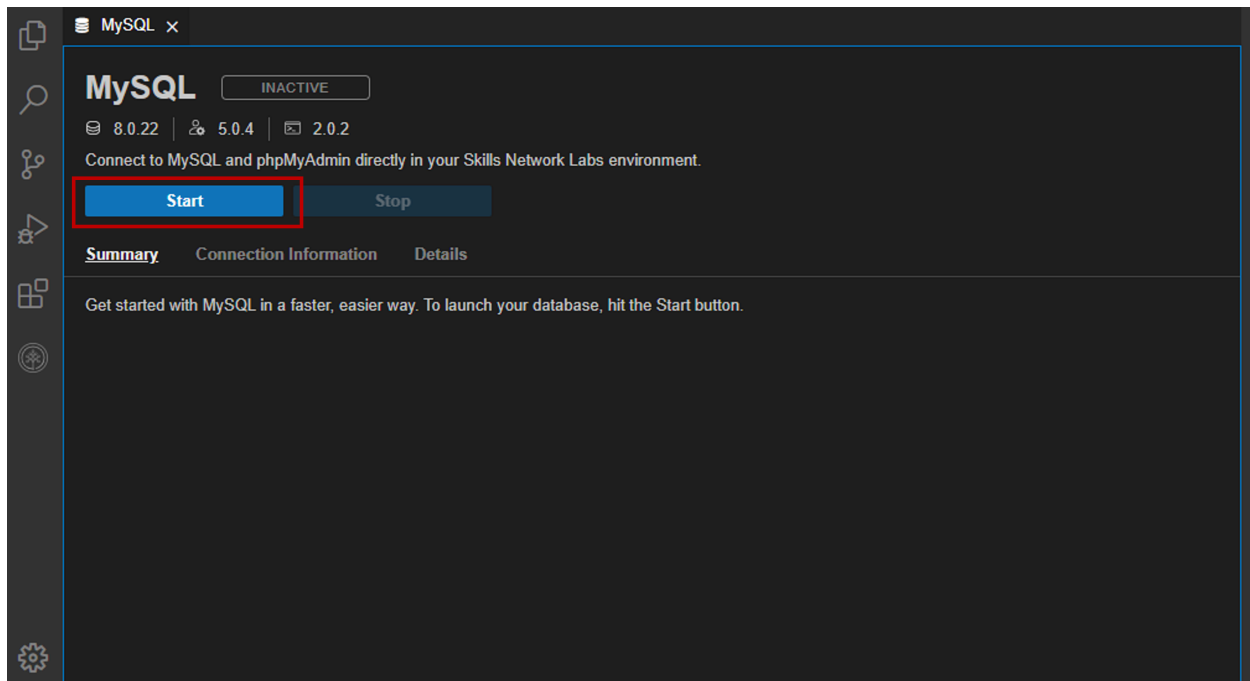
```
[2024-06-12 02:19:51,129] INFO [BrokerServer id=1] Transition from STARTING to STARTED (kafka.server.BrokerServer)
[2024-06-12 02:19:51,130] INFO Kafka version: 3.7.0 (org.apache.kafka.common.utils.AppInfoParser)
[2024-06-12 02:19:51,135] INFO Kafka commitId: 2ae524ed625438c5 (org.apache.kafka.common.utils.AppInfoParser)
[2024-06-12 02:19:51,135] INFO Kafka startTimeMs: 1718173191129 (org.apache.kafka.common.utils.AppInfoParser)
[2024-06-12 02:19:51,137] INFO [KafkaRaftServer nodeId=1] Kafka Server started (kafka.server.KafkaRaftServer)
[2024-06-12 02:20:25,678] INFO [ReplicaFetcherManager on broker 1] Removed fetcher for partitions Set(bankbranch-1, bankbranch-0) (kafka.server.ReplicaFetcherManager)
[2024-06-12 02:20:25,718] INFO [LogLoader partition=bankbranch-1, dir=/tmp/kraft-combined-logs] Loading producer state till offset 0 with message format version 2 (kafka.log.UnifiedLog$)
[2024-06-12 02:20:25,722] INFO Created log for partition bankbranch-1 in /tmp/kraft-combined-logs/bankbranch-1 with properties {} (kafka.log.LogManager)
[2024-06-12 02:20:25,725] INFO [Partition bankbranch-1 broker=1] No checkpointed highwatermark is found for partition bankbranch-1 (kafka.cluster.Partition)
[2024-06-12 02:20:25,727] INFO [Partition bankbranch-1 broker=1] Log loaded for partition bankbranch-1 with initial high watermark 0 (kafka.cluster.Partition)
[2024-06-12 02:20:25,745] INFO [LogLoader partition=bankbranch-0, dir=/tmp/kraft-combined-logs] Loading producer state till offset 0 with message format version 2 (kafka.log.UnifiedLog$)
[2024-06-12 02:20:25,746] INFO Created log for partition bankbranch-0 in /tmp/kraft-combined-logs/bankbranch-0 with properties {} (kafka.log.LogManager)
```

```
(org.apache.kafka.server.log.remote.storage.RemoteLogManagerConfig)
[2024-11-19 04:38:18,606] INFO [BrokerServer id=1] Waiting for the broker to be unfenced (kafka.server.BrokerServer)
[2024-11-19 04:38:18,655] INFO [BrokerLifecycleManager id=1] The broker has been unfenced. Transitioning from RECOVERY to (kafka.server.BrokerLifecycleManager)
[2024-11-19 04:38:18,656] INFO [BrokerServer id=1] Finished waiting for the broker to be unfenced (kafka.server.BrokerServer)
[2024-11-19 04:38:18,657] INFO authorizerStart completed for endpoint PLAINTEXT. Endpoint is now READY. (org.apache.kafka.network.EndpointReadyFutures)
[2024-11-19 04:38:18,657] INFO [SocketServer listenerType=BROKER, nodeId=1] Enabling request processing. (kafka.network.SocketServer)
[2024-11-19 04:38:18,658] INFO Awaiting socket connections on 0.0.0.0:9092. (kafka.network.DataPlaneAcceptor)
[2024-11-19 04:38:18,659] INFO [BrokerServer id=1] Waiting for all of the authorizer futures to be completed (kafka.server.BrokerServer)
[2024-11-19 04:38:18,659] INFO [BrokerServer id=1] Finished waiting for all of the authorizer futures to be completed (kafka.server.BrokerServer)
[2024-11-19 04:38:18,659] INFO [BrokerServer id=1] Waiting for all of the SocketServer Acceptors to be started (kafka.server.BrokerServer)
[2024-11-19 04:38:18,659] INFO [BrokerServer id=1] Finished waiting for all of the SocketServer Acceptors to be started (kafka.server.BrokerServer)
[2024-11-19 04:38:18,659] INFO [BrokerServer id=1] Transition from STARTING to STARTED (kafka.server.BrokerServer)
[2024-11-19 04:38:18,659] INFO Kafka version: 3.8.0 (org.apache.kafka.common.utils.AppInfoParser)
[2024-11-19 04:38:18,660] INFO Kafka commitId: 771b9576b00ecf5b (org.apache.kafka.common.utils.AppInfoParser)
[2024-11-19 04:38:18,660] INFO Kafka startTimeMs: 1732009098659 (org.apache.kafka.common.utils.AppInfoParser)
[2024-11-19 04:38:18,661] INFO [KafkaRaftServer nodeId=1] Kafka Server started (kafka.server.KafkaRaftServer)
```

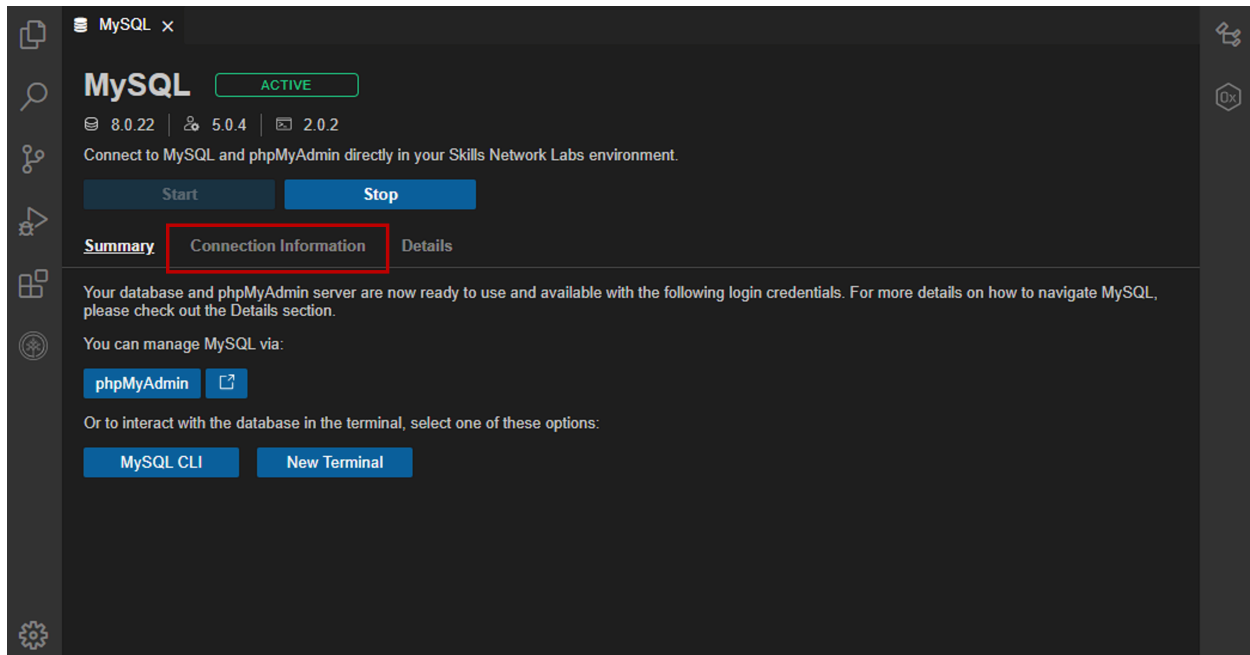
Exercise 3: Start MySQL server and setup the database

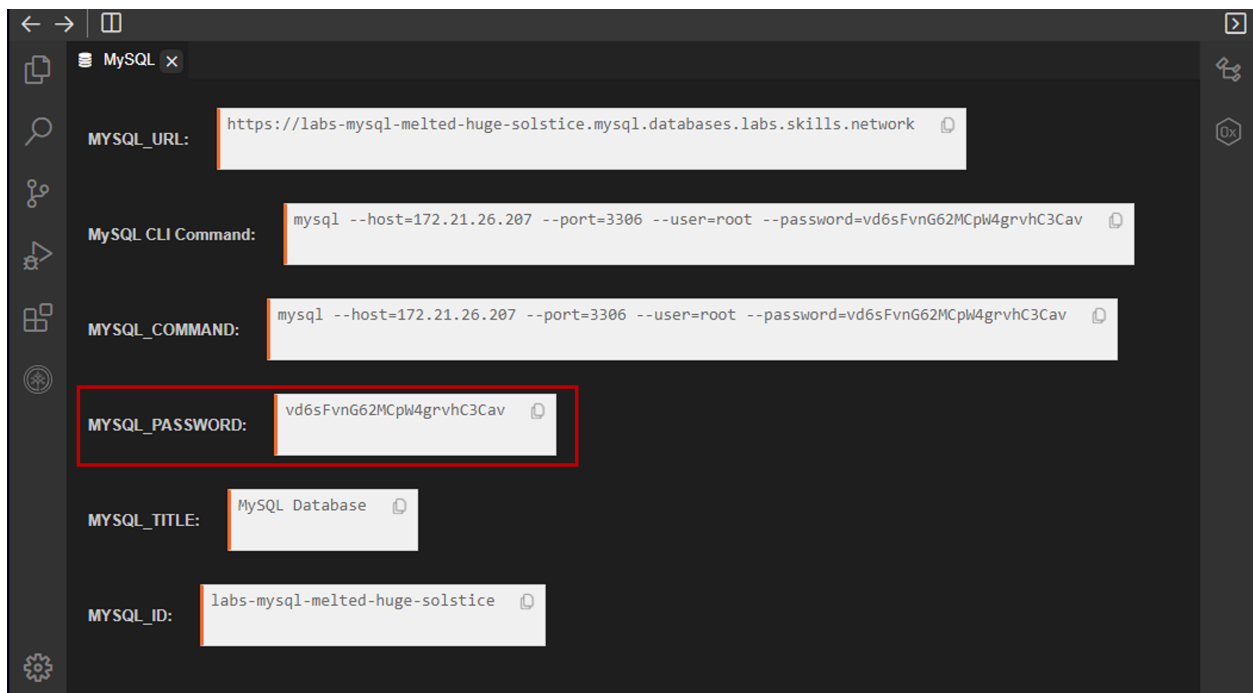
Open MySQL Page in IDE

1. On the launching page, click the **Start** button.



1. Once the MySQL server started, select the **Connection Information** tab. From that, copy the password.





1. Connect to the MySQL server using the command below in the terminal. Make sure you use the password given to you when the MySQL server starts. Please make a note of the password because you will need it later.

```
mysql --host=mysql --port=3306 --user=root --password=Replace your password
```

```
mysql --host=172.21.189.194 --port=3306 --user=root --password=lTcCS6cEqLwtHivxe
```

```
theia@theiadocker-sannyjrgarin: /home/project/kafka_2.13-3.8.0 X theia@theiadocker-sannyjrgarin: /home/project X
theia@theiadocker-sannyjrgarin:/home/project$ mysql --host=172.21.189.194 --port=3306 --user=root --password=1T
cCS6cEqLwtHivxeCbrfntR
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 719
Server version: 8.0.37 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> 
```

4. Create a database named `tolldata`.

At the `mysql>` prompt, run the command below to create the database.

```
1 create database tolldata;
```

```
mysql> create database tolldata;
Query OK, 1 row affected (0.01 sec)

mysql> 
```

5. Create a table named `livetolldata` with the schema to store the data generated by the traffic simulator.

Run the following command to create the table:

```
1 use tolldata;
2
3 create table livetolldata(timestamp datetime,vehicle_id int,vehicle_type char(15),toll_plaza_id smallint);
```

Note: This is the table where you will store all streamed data that comes from Kafka. Each row is a record of when a vehicle has passed through a certain toll plaza along with its type and anonymized id.

```
mysql> use tolldata;
Database changed
mysql>
mysql> create table livetolldata(timestamp datetime,vehicle_id int,vehicle_type char(15),toll_plaza_id smallint);
Query OK, 0 rows affected (0.02 sec)

mysql> 
```

6. Disconnect from the MySQL server.

```
1 exit
```


Exercise 4: Install the Python packages

1. Install the Python module `kafka-python`. This Python module will help you to communicate with kafka server. It can used to send and receive messages from Kafka.

```
1 pip3 install kafka-python
```

2. Install the Python module `mysql-connector-python` using the `pip` command.

```
1 pip3 install mysql-connector-python==8.0.31
```

This Python module will help you to interact with MySQL server.

```
theia@theiadocker-sannyjrgarin:/home/project$ pip3 install kafka-python
Defaulting to user installation because normal site-packages is not writeable
Collecting kafka-python
  Downloading kafka_python-2.0.2-py2.py3-none-any.whl (246 kB)
----- 246.5/246.5 KB 6.9 MB/s eta 0:00:00
Installing collected packages: kafka-python
Successfully installed kafka-python-2.0.2
theia@theiadocker-sannyjrgarin:/home/project$
```

```
theia@theiadocker-sannyjrgarin:/home/project$ pip3 install mysql-connector-python==8.0.31
Defaulting to user installation because normal site-packages is not writeable
Collecting mysql-connector-python==8.0.31
  Downloading mysql_connector_python-8.0.31-cp310-cp310-manylinux1_x86_64.whl (23.5 MB)
----- 23.5/23.5 MB 63.1 MB/s eta 0:00:00
Collecting protobuf<=3.20.1,>=3.11.0
  Downloading protobuf-3.20.1-cp310-cp310-manylinux2_12_x86_64.whl (1.1 MB)
----- 1.1/1.1 MB 76.5 MB/s eta 0:00:00
Installing collected packages: protobuf, mysql-connector-python
Successfully installed mysql-connector-python-8.0.31 protobuf-3.20.1
```

Exercise 5: Create data pipeline for toll data

1. Create a Kafka topic named `toll`.
2. Download the `toll_traffic_generator.py` from the url given below using `wget`.

```
1 wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DB0250EN-SkillsNetwork/labs/Final%20Assignment/toll_traffic_generator.py
```

3. Open the code using the editor using the "Menu -> File -> Open" option.
4. Open the `toll_traffic_generator.py` and set the topic to `toll`.
5. Run the `toll_traffic_generator.py`.

Create a `toll_traffic_generator.py` file that creates a topic named `toll`

This file will generate a data that mimic a streaming data from a toll record data.

```
"""
Top Traffic Simulator
"""
from time import sleep, time, ctime
from random import random, randint, choice
from kafka import KafkaProducer
producer = KafkaProducer(bootstrap_servers='localhost:9092')

TOPIC = 'toll'

VEHICLE_TYPES = ("car", "car", "car", "car", "car", "car", "car", "car",
                 "car", "car", "car", "truck", "truck", "truck",
                 "truck", "van", "van")
for _ in range(100000):
    vehicle_id = randint(10000, 100000000)
    vehicle_type = choice(VEHICLE_TYPES)
    now = ctime(time())
    plaza_id = randint(4000, 4010)
    message = f"{now},{vehicle_id},{vehicle_type},{plaza_id}"
    message = bytearray(message.encode("utf-8"))
    print(f"A {vehicle_type} has passed by the toll plaza {plaza_id} at {now}.")
    producer.send(TOPIC, message)
    sleep(random() * 2)
```

Run the `toll_traffic_generator.py`

```
22 producer.send(topic, message)

heiadocker-sannyjrgarin: /home/project x theia@theiadocker-sannyjrgarin: /home/project x theia@theiadocker-sannyjrgarin: /home/project x

theia@theiadocker-sannyjrgarin:/home/project$ python3 toll_traffic_generator.py
A car has passed by the toll plaza 4009 at Tue Nov 19 05:03:04 2024.
A car has passed by the toll plaza 4002 at Tue Nov 19 05:03:06 2024.
A van has passed by the toll plaza 4009 at Tue Nov 19 05:03:07 2024.
A truck has passed by the toll plaza 4000 at Tue Nov 19 05:03:08 2024.
A car has passed by the toll plaza 4003 at Tue Nov 19 05:03:09 2024.
A car has passed by the toll plaza 4006 at Tue Nov 19 05:03:09 2024.
A car has passed by the toll plaza 4007 at Tue Nov 19 05:03:10 2024.
A car has passed by the toll plaza 4005 at Tue Nov 19 05:03:11 2024.
A car has passed by the toll plaza 4006 at Tue Nov 19 05:03:12 2024.
█
```

```
python3 toll_traffic_generator.py
```

6. Download the `streaming-data-reader.py` from the URL below using `wget`.

```
1 wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/vVxmU5uatDowVAIKRZrFjg/streaming-data-reader.py
```

7. Open the `streaming-data-reader.py` and modify the following details so that the program can connect to your MySQL server.

TOPIC

DATABASE

USERNAME

PASSWORD

8. Run the `streaming-data-reader.py`.

```
1 python3 streaming-data-reader.py
```

9. If you completed all the steps correctly, the streaming toll data will get stored in the table `livetolldata`. As a last step in this lab, open mysql CLI and list the top 10 rows in the table `livetolldata`.

Create a file `streaming-data-reader.py` that will connect to the MySQL database and connect to the Kafka topic and listen to the streaming data as a consumer and saved the data into the MySQL database.

```
"""
```

Streaming data consumer

```
"""
```

```

from datetime import datetime
from kafka import KafkaConsumer
import mysql.connector

TOPIC='toll'
DATABASE = 'tolldata'
USERNAME = 'root'
PASSWORD = 'ITcCS6cEqLwtHivxeCbrfntr'

print("Connecting to the database")
try:
    connection = mysql.connector.connect(host='mysql', database=DATABASE,
user=USERNAME, password=PASSWORD)
except Exception:
    print("Could not connect to database. Please check credentials")
else:
    print("Connected to database")
cursor = connection.cursor()

print("Connecting to Kafka")
consumer = KafkaConsumer(TOPIC)
print("Connected to Kafka")
print(f"Reading messages from the topic {TOPIC}")
for msg in consumer:

    # Extract information from kafka

    message = msg.value.decode("utf-8")

    # Transform the date format to suit the database schema
    (timestamp, vehcile_id, vehicle_type, plaza_id) = message.split(",")

    dateobj = datetime.strptime(timestamp, '%a %b %d %H:%M:%S %Y')
    timestamp = dateobj.strftime("%Y-%m-%d %H:%M:%S")

    # Loading data into the database table

```

```
sql = "insert into livetolldata values(%s,%s,%s,%s)"
result = cursor.execute(sql, (timestamp, vehcile_id, vehicle_type, plaza_id))
print(f"A {vehicle_type} was inserted into the database")
connection.commit()
connection.close()
```

```
python3 streaming-data-reader.py
```



Now, the streaming toll data will get stored in the table `livetolldata` . As a last task, open mysql CLI and list the top 10 rows in the table `livetolldata` .



As you can see the data that was manufactured by the python script was assigned to a Kafka-producer in Apache Kafka Instance, then saved to MySQL server database. This is just a sample of a live streaming data, in real life it could be more complex than this.

In this project you've learned:

- Start a MySQL database server
- Create a table to hold the toll data
- Start the Kafka server
- Install the Kafka Python driver
- Install the MySQL Python driver
- Create a topic named toll in Kafka
- Download streaming data generator program
- Customize the generator program to stream to toll topic
- Download and customize streaming data consumer
- Customize the consumer program to write into a MySQL database table
- Verify that streamed data is being collected in the database table